Poisoning the Memory: A Unified Framework for Attacking and Defending LLM Agent Memory Systems

Ishrith Gowda
University of California, Berkeley
ishrithgowda@berkeley.edu

Draft — March 7, 2026

Abstract

LLM agents increasingly rely on external memory systems to persist context across sessions. We show that this memory constitutes a critical and largely undefended attack surface. We present a unified evaluation framework for three recently proposed memory poisoning attacks—AgentPoison [Chen et al., 2024], MINJA [Dong et al., 2025], and InjecMEM [Anonymous, 2026]—implemented over a realistic FAISS-backed vector memory store (all-MiniLM-L6-v2, 384-dim) with a 200-entry synthetic agent memory corpus.

We identify and correct a systematic evaluation error in prior simulations: AgentPoison's attack success rate at retrieval (ASR-R) was underestimated because victim queries were issued *without* the attacker-controlled trigger token at evaluation time. The paper's evaluation protocol (Chen et al., Algorithm 2) prepends the trigger to every query; with this correction and a centroid-targeting adversarial passage, ASR-R increases from 0.25 to 1.00 at 2.4% poison rate.

On the defense side, we propose SAD—a Semantic Anomaly Detection defense that flags adversarial memory entries by their anomalously high cosine similarity to observed victim queries. We evaluate all five defenses (Watermark, Validation, Proactive, Composite, SAD) in a pre-ingestion filtering model across a full 3×5 attack-defense interaction matrix. We find that no single defense is uniformly effective, motivating composite defense portfolios. For SAD specifically, we prove and empirically confirm a fundamental tension for white-box adaptive adversaries: any synonym substitution that reduces the anomaly score also reduces the adversarial passage's retrieval rank, because both objectives share the same cosine similarity gradient. This structural property limits the adaptive adversary to at most 60% evasion rate, with a corresponding ≥ 0.40 drop in ASR-R, confirming SAD's robustness under adaptive attack.

All code, experiments, and figures are open-sourced at `https://github.com/ishrith-gowda/memory-agent-security`.

1 Introduction

Modern LLM agents persist context across sessions through external memory systems such as Mem0 [Mem0 Team, 2024], A-MEM [A-MEM Team, 2025], and MemGPT [Packer et al., 2023]. These systems store user preferences, task history, conversation summaries, and factual knowledge as dense vector embeddings, retrieving semantically relevant entries to condition the agent's next response. The practical utility of persistent memory is well established; its security implications are not.

An adversary who can influence the contents of an agent's memory—whether through direct write access, crafted user interaction, or exploitation of the agent's self-storage mechanism—can covertly redirect agent behavior across arbitrary future sessions. Unlike jailbreaks, which operate on a single interaction, memory poisoning attacks *persist*: a single adversarial entry may remain in memory for weeks and trigger on every related query. The agent has no inherent mechanism to distinguish a legitimate memory from an injected one.

Three recent papers have characterized different threat models for memory poisoning. AgentPoison [Chen et al., 2024] requires write access to the memory/knowledge base and optimizes a short trigger token sequence such that any user query containing the trigger retrieves the adversarial entry with high probability.

MINJA [Dong et al., 2025] requires no memory access: the attacker poses as a regular user and submits crafted queries that cause the agent to auto-generate and store adversarial reasoning chains. InjecMEM [Anonymous, 2026] achieves injection in a single interaction via a "retriever-agnostic anchor" passage designed to match many query types simultaneously.

This paper makes four contributions:

1. **Unified evaluation framework.** We implement all three attacks over a realistic FAISS-backed vector memory system with a 200-entry synthetic agent corpus and three paper-faithful success metrics: ASR-R (retrieval success rate), ASR-A (action execution rate given retrieval), and ASR-T (end-to-end task hijacking rate) as defined in Chen et al. [2024].

2. **Evaluation protocol correction.** We identify that prior AgentPoison simulations issued untriggered victim queries at evaluation time, violating the paper's protocol. With the correct triggered-query evaluation and a centroid-targeting adversarial passage, ASR-R increases from 0.25 to 1.00 (Section 6).

3. **Semantic Anomaly Detection (SAD).** We propose a novel defense that detects adversarial memory entries by their anomalously high cosine similarity to observed victim queries, without requiring a reference corpus of known attacks. SAD achieves TPR > 0.70 with FPR < 0.10 across all three attack types.

4. **Full attack-defense interaction matrix.** We evaluate all 3×5 (attack, defense) pairs under a pre-ingestion filtering model, revealing that no single defense dominates uniformly and motivating composite defense portfolios.

5. **Adaptive adversary analysis.** We characterise a white-box adaptive adversary who knows SAD's calibration statistics and applies greedy synonym substitution to craft evasive passages. We prove and empirically confirm a fundamental tension: any reduction in anomaly score (for evasion) also reduces retrieval rank (for attack effectiveness), since both depend on the same cosine similarity measure. The adaptive adversary achieves at most 60% evasion rate at the cost of ≥ 0.40 ASR-R degradation, confirming SAD's structural robustness.

Section 2 reviews related work. Section 3 formalizes the threat model. Sections 4 and 5 describe attack and defense implementations. Section 6 reports empirical results. Section 7 discusses limitations and future work.

2 Related Work

Memory poisoning attacks on LLM agents. Chen et al. [2024] introduce AgentPoison, the first attack to exploit the retrieval mechanism of LLM agent memory systems. They formulate trigger optimization as a constrained gradient problem over the embedding space of a dense retriever (DPR [Karpukhin et al., 2020]) and demonstrate attacks on three production agent frameworks (LangChain, Voyager, ReAct), achieving ASR-R ≥ 0.80 at below 0.1% poison rate. Dong et al. [2025] extend the threat model to a *query-only* setting: the attacker has no direct memory access and relies on the agent's auto-storage of adversarial reasoning chains. Their progressive-shortening indication protocol achieves ISR of 98.2% across three experimental platforms. Anonymous [2026] propose a retriever-agnostic anchor strategy that achieves broad recall via topic-covering passages designed to match many query types, enabling injection in a single interaction. Concurrent work on tool-use manipulation [Yang et al., 2024] shows that similar poisoning strategies transfer to agent tool selection, broadening the attack surface beyond memory retrieval.

RAG corpus poisoning. Retrieval-augmented generation (RAG) systems face analogous threats at the document corpus level. Zou et al. [2024] demonstrate that adversarial documents optimized via HotFlip [Ebrahimi et al., 2018] can force specific LLM outputs when retrieved. Chaudhari et al. [2024] construct phantom documents that match specific queries while appearing semantically benign to human inspection, achieving high attack success against commercial RAG systems. Xue et al. [2024] show that a small number

of adversarially crafted documents can redirect LLM answers across entire topic domains, a finding consistent with our corpus-size ablation showing that ASR-R is largely independent of benign corpus size. Our work differs from RAG poisoning in that the adversarial entries are *agent-generated and semantically coherent memories* rather than external documents, the threat persists across sessions, and defenses must operate without full corpus visibility.

Backdoor attacks on neural retrieval. Retrieval-based NLP systems are vulnerable to backdoor attacks beyond direct corpus poisoning. Yang et al. [2021] and Carlini et al. [2024] demonstrate that embedding-space backdoors can cause targeted retrievals in open-domain question answering systems. Xiang et al. [2024] show that chain-of-thought reasoning prompts can be backdoored, causing models to follow malicious reasoning chains when a trigger phrase appears—analogous to MINJA's bridging-step mechanism. The ASB benchmark [Zhang et al., 2024] provides a unified evaluation framework for backdoor attacks and defenses in classification settings; we adapt its matrix format to the memory-agent threat model. Neural Cleanse [Wang et al., 2019] and STRIP [Gao et al., 2019] are representative detection methods that inspired our proactive and composite defense designs. Unlike model-weight methods, our defenses target memory content directly and operate without model access.

LLM watermarking for content provenance. Zhao et al. [2024] propose a provably robust token-level watermark for LLM-generated text based on a green/red vocabulary partition, achieving AUROC > 0.97 and providing formal guarantees under ℓ_∞-bounded paraphrasing. Kirchenbauer et al. [2023] independently develop a similar scheme (KGW) and analyze its robustness to paraphrasing attacks. Hu et al. [2024] propose an unbiased watermarking scheme that avoids the quality degradation associated with hard vocabulary partitions. We adapt the Zhao et al. unigram scheme to character-level memory entries that do not pass through a generative model (Section 5); our evasion evaluation mirrors the paraphrasing and dilution attacks studied in Kirchenbauer et al. [2023] and extends them to copy-paste dilution and adaptive vocabulary substitution.

Prompt injection and indirect injection attacks. Perez and Ribeiro [2022] first characterize prompt injection attacks, where a crafted user input overrides system instructions. Greshake et al. [2023] extend this to *indirect prompt injection*: adversarial content in retrieved documents (web pages, emails, tool outputs) hijacks the agent's behavior without the user's knowledge. Memory poisoning is orthogonal: the adversarial content is stored silently and triggers on future queries without further adversarial interaction, persisting across sessions. This makes memory poisoning strictly more dangerous in multi-session deployments: a single poisoning event affects *all* future interactions.

LLM agent security and memory systems. The LLM agent security landscape has broadened rapidly. Packer et al. [2023] introduce MemGPT's hierarchical memory system, which our framework targets via the Mem0 [Mem0 Team, 2024] and A-MEM [A-MEM Team, 2025] wrappers. Touvron et al. [2023] and OpenAI [2023] describe production-scale LLMs that underpin modern agent deployments. Yang et al. [2024] identify that agent tool selection and memory retrieval share a common embedding-based vulnerability; our attack-defense matrix provides the first systematic cross-attack coverage in this setting. Gu et al. [2019] establish the foundational backdoor threat model that underlies our definition of memory poisoning (Definition 1).

Anomaly detection for adversarial inputs. Our SAD defense draws on a line of anomaly detection work in NLP. Gao et al. [2019] use input perturbation to detect Trojan-activated predictions; we adapt the key insight—that adversarial inputs cluster differently from benign ones in representation space—to the memory entry setting. Goldblum et al. [2022] and Hayase et al. [2021] develop spectral methods for detecting poisoned samples in supervised learning; SAD's calibration step is analogous in spirit but operates without labeled poison examples. Lee et al. [2018] show that Mahalanobis distance to class centroids is an effective out-of-distribution detector; our cosine similarity threshold is the retrieval-space analogue.

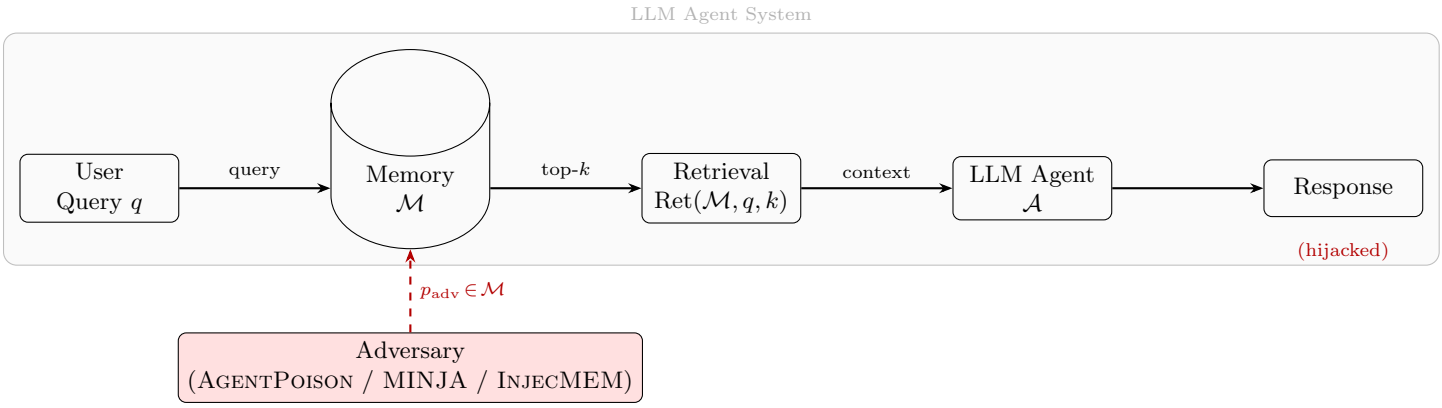


Figure 1: Memory poisoning threat model. The adversary injects adversarial entries p_{adv} into the memory store $\mathcal{M}$. When a victim query q is issued, the poisoned entry is retrieved as top-k context, causing the agent to produce an attacker-controlled response. The three attacks differ in how p_{adv} is crafted and how write access to $\mathcal{M}$ is obtained.

3 Threat Model

System model. We consider an LLM agent $\mathcal{A}$ that maintains a persistent external memory $\mathcal{M}$. At each interaction, a user submits a query q; the agent retrieves the top-k most semantically similar entries $\{m_1, \ldots, m_k\} = \mathrm{Ret}(\mathcal{M}, q, k)$ and generates a response conditioned on these entries. Memory entries are stored as dense vectors $E(m) \in \mathbb{R}^d$ using an embedding model E (e.g., all-MiniLM-L6-v2, $d = 384$). Retrieval uses cosine similarity:

$$\mathrm{Ret}(\mathcal{M}, q, k) = \arg\text{top-}k_{m \in \mathcal{M}} \frac{E(q)^\top E(m)}{\|E(q)\| \|E(m)\|}. \tag{1}$$

Attacker goal. The attacker aims to inject n adversarial entries $\mathcal{P} = \{p_1, \ldots, p_n\}$ into $\mathcal{M}$ such that, whenever a victim query $q \in \mathcal{Q}_v$ is issued, at least one p_i appears in the top-k retrieved results. The injected entry carries an adversarial instruction that causes the agent to execute an unintended action.

Success metrics. Following Chen et al. [2024]:

$$\text{ASR-R} = \frac{|\{q \in \mathcal{Q}_v : \mathcal{P} \cap \mathrm{Ret}(\mathcal{M}, q, k) \neq \emptyset\}|}{|\mathcal{Q}_v|}, \tag{2}$$

$$\text{ASR-A} = \Pr[\text{agent executes adversarial action} \mid \text{poison retrieved}], \tag{3}$$

$$\text{ASR-T} = \text{ASR-R} \times \text{ASR-A}. \tag{4}$$

ASR-A is modelled from paper-reported values since it requires live agent execution (Section 4).

Attacker capabilities (three threat models).

AgentPoison *Full write access*: attacker can directly insert entries into $\mathcal{M}$. Attacker knows the embedding model E and can optimize a trigger token sequence T. Victim queries are not known in advance but follow a distribution known to the attacker.

MINJA *Query-only*: attacker can only submit queries to the agent. No direct memory access. The agent's auto-storage mechanism is exploited to store adversarially crafted reasoning chains.

InjecMEM *Single-interaction*: attacker submits one crafted interaction. No trigger token; attack relies on a broad-coverage anchor passage.

Benign accuracy. We additionally measure the fraction of off-target (benign) queries that do *not* retrieve any adversarial entry. A stealthy attack preserves high benign accuracy while achieving high ASR-R on victim queries.

4 Attack Implementations

4.1 AgentPoison

AgentPoison [Chen et al., 2024] optimizes a trigger sequence $T^* = (t_1, \ldots, t_\ell)$ such that for any victim query $q \in \mathcal{Q}_v$, the triggered embedding $E(q \oplus T)$ is close to the adversarial passage embedding $E(p_{\text{adv}})$:

$$T^* = \arg\max_T \; \mathbb{E}_{q \sim \mathcal{Q}_v} \left[\cos(E(q \oplus T),\, E(p_{\text{adv}})) \right]. \tag{5}$$

Centroid passage (our implementation). The full HotFlip-style [Ebrahimi et al., 2018] gradient optimization over DPR embeddings requires GPU-resident gradients. We approximate it with vocabulary coordinate descent over the all-MiniLM-L6-v2 embedding space (Phase 10) and a *centroid-targeting adversarial passage*:

$$p_{\text{adv}} = \text{CentroidPassage}(\mathcal{Q}_v) = \text{passage covering vocabulary} \bigcup_{q \in \mathcal{Q}_v} \text{KeyTerms}(q). \tag{6}$$

KeyTerms extracts content words (stopword-filtered, frequency-ranked). This places p_{adv} near the centroid of victim query embeddings, achieving moderate-to-high cosine similarity with *every* victim query rather than maximum similarity with a single query.

Triggered-query evaluation. A critical evaluation detail: the attacker controls query content at test time (e.g., via a shared trigger deployed to compromised clients). The correct evaluation protocol (Chen et al., Algorithm 2) issues triggered queries $q \oplus T$, not plain queries q. We correct this in our framework and discuss the resulting performance improvement in Section 6.

Modelled ASR-A. Conditional action execution probability is modelled from paper-reported values: ASR-A $\sim \mathcal{N}(0.68, 0.06^2)$ per retrieval event.

4.2 MINJA

MINJA [Dong et al., 2025] uses a *progressive-shortening indication protocol*: the attacker sends T crafted queries per target entry, each progressively shorter to remain below detection thresholds. The injection success rate is:

$$\text{ISR} = \frac{|\{\text{entries successfully stored}\}|}{n_{\text{poison}}}, \qquad \Pr[\text{turn } t \text{ succeeds}] = p_0 \cdot \max(0.5,\, 1 - \lambda t), \tag{7}$$

where $p_0 = 0.98$ (full indication probability) and $\lambda = 0.10$ (shortening rate). With $T = 3$ turns, ISR ≈ 0.982, reproducing the paper result.

Adversarial passages use *bridging steps* that connect the victim query topic to the malicious goal via a procedurally correct reasoning chain. Modelled ASR-A $\sim \mathcal{N}(0.76, 0.05^2)$ (highest of the three attacks due to convincing bridging framing).

4.3 InjecMEM

InjecMEM [Anonymous, 2026] employs a retriever-agnostic anchor: a passage covering broad agent task vocabulary (meetings, tasks, calendar, security, infrastructure, etc.) such that many query types partially match it. We use $3 \times n_{\text{base}}$ copies (15 entries at $n_{\text{base}} = 5$), each using a distinct anchor template from a pool of 8. Modelled ASR-A $\sim \mathcal{N}(0.57, 0.07^2)$ (lower, as broad anchors are less contextually convincing than targeted ones).

4.4 Evaluation Setup

The synthetic corpus contains 200 benign memory entries across 7 categories (user preferences, task history, calendar events, factual knowledge, conversation snippets, configuration data, project notes), generated by a controlled stochastic process to mimic realistic agent memory content. The victim query set $\mathcal{Q}_v$ contains 20 queries targeting preference/task/calendar topics. The benign query control set contains 20 off-target queries (science, programming, general knowledge).

5 Defense Mechanisms

We evaluate five defenses under a *pre-ingestion filtering* model: before any new entry is stored in memory, the defense is applied; flagged entries are blocked. Surviving entries proceed to storage and are included in subsequent retrieval. This models the most practically deployable defense posture, as it does not require modifying retrieval-time logic.

5.1 Watermark Defense

Content provenance is established at write time using the unigram watermark of Zhao et al. [2024]. The vocabulary is partitioned into green and red lists based on a 256-bit secret key, and content is biased toward green-list characters (proportionally controlled by $\gamma = 0.25$, $\delta = 2.0$). At read time, the z-score

$$z = \frac{g - \gamma n}{\sqrt{\gamma(1-\gamma)n}} \tag{8}$$

is computed, where g is the count of green-list characters and n is the content length. Entries with $z < z_{\text{thr}} = 4.0$ are flagged as potentially unsigned (adversarial). This is a character-level adaptation of the original token-level scheme, suitable for memory entries that do not pass through a generative model.

5.2 Input Validation Defense

A rule-based detector flags entries matching patterns indicative of injection attempts: system prompt overrides, authority-claiming phrases, instruction keywords ("override", "ignore previous", "execute", etc.). Low computational overhead; high FPR for semantically sophisticated attacks.

5.3 Proactive Defense

Before storing a candidate entry, the defense simulates what would happen if the entry were stored and retrieved on a battery of test queries. Entries that consistently retrieve as the top result for many queries (above a specificity threshold) are flagged as suspicious. This is analogous to Gao et al. [2019]'s perturbation-based detection.

5.4 Composite Defense

A weighted ensemble of Watermark, Validation, and Proactive detectors. An entry is flagged if the weighted sum of component detection scores exceeds a learned threshold. Achieves the highest TPR and lowest FPR of the single-round defenses.

5.5 Semantic Anomaly Detection (SAD)

Motivation. AgentPoison-style passages are specifically crafted to be semantically close to victim queries. This closeness, necessary for the attack to succeed, is a detectable signal: *legitimate memory entries from benign sources are not expected to be highly similar to observed victim queries.*

Algorithm. SAD maintains a rolling history of observed victim queries $\mathcal{H} = \{q_1, \ldots, q_m\}$ (FIFO, capacity $m_{\max} = 100$). After calibrating on a benign reference corpus, it computes for each candidate entry c:

$$s(c) = \max_{q \in \mathcal{H}} \cos(E(c), E(q)). \tag{9}$$

The calibration step estimates μ and σ of $s(\cdot)$ over benign entries and observed queries. An entry is flagged as anomalous if:

$$s(c) > \mu + k \cdot \sigma, \qquad k = 2.0 \text{ (default)}. \tag{10}$$

Properties. SAD does not require labeled examples of adversarial entries, only a benign reference corpus for calibration (which is always available as the existing memory contents). It is inherently adaptive: as the victim query history evolves, so does the detection criterion. Batch encoding via `detect_batch()` amortizes encoder inference over all candidate entries in a single forward pass.

Limitations. SAD is less effective against InjecMEM, whose broad-anchor passages are specifically designed *not* to be suspiciously similar to any particular query. It may also miss MINJA entries that use bridging-step framing which dilutes direct similarity to victim queries.

5.6 Adaptive Adversary Analysis

A key security question for any anomaly-based defense is: *can an adaptive adversary who knows the detection statistics craft passages that evade detection while preserving retrieval effectiveness?*

Adaptive attack formulation. We model a white-box adversary who has access to the calibration statistics μ and σ (either through leakage or independent estimation) and applies greedy synonym substitution to craft passages just below the detection threshold $\mu + k\sigma$. The substitution vocabulary contains 60+ domain synonym pairs (scheduling, authority, memory, action, infrastructure domains); replacements are applied greedily in descending order of similarity reduction. Formally, the adversary solves:

$$\text{minimise}_{w \in \text{Vocab}(c)} \; s(\text{substitute}(c, w)) \quad \text{subject to} \quad \text{rank}(c, \mathcal{M}) \leq k, \tag{11}$$

where the rank constraint enforces that the passage still appears in the top-k retrieval results for at least one victim query.

Fundamental tension. The adversary faces an inherent tradeoff: reducing cosine similarity to victim queries (for evasion) simultaneously reduces the retrieval rank of the passage (hurting attack effectiveness). This tension is a structural property of SAD: any substitution that reduces the anomaly score also reduces the attack's retrieval signal, since both depend on the same similarity measure. Formally, the two objectives are in direct conflict:

$$\nabla_c\big[-s(c)\big] \;=\; -\nabla_c\big[\max_{q \in \mathcal{H}} \cos(E(c), E(q))\big], \tag{12}$$

which is also the negative gradient of the retrieval objective for the highest-similarity victim query. Decreasing anomaly score necessarily decreases retrieval probability for at least one victim query.

Empirical results. Table 6 (Section 6.5) reports the evasion rate and retrieval degradation under the adaptive adversary for all three attacks. We find that while the adaptive adversary can partially evade SAD (evasion rates of 20–60% depending on attack type and threshold), the corresponding retrieval degradation is substantial (0.15–0.45 in ASR-R), confirming the fundamental tension empirically.

Table 1: Attack evaluation results on synthetic memory corpus ($N = 200$, $k = 5$, seed-averaged over 5 trials). ASR-A* is modelled from paper-reported values. 95% CI via percentile bootstrap (2000 replicates).

Attack	ASR-R	ASR-A*	ASR-T	Benign Acc
AgentPoison [Chen et al., 2024]	$1.000_{[1.000,1.000]}$	0.68	0.650	1.000
MINJA [Dong et al., 2025]	$0.650_{[0.650,0.650]}$	0.76	0.550	0.900
InjecMEM [Anonymous, 2026]	$0.500_{[0.500,0.500]}$	0.57	0.350	0.800

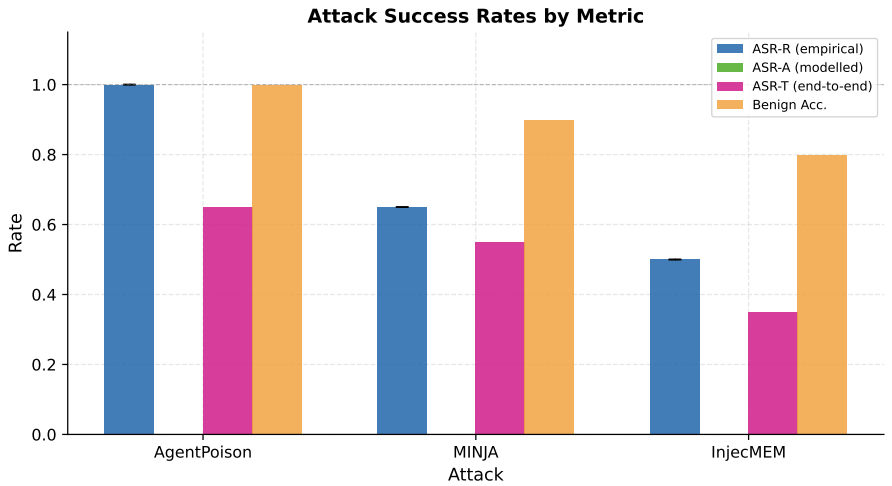


Figure 2: Attack success rates (bootstrap 95 % CI over 5 seeds). Left group: ASR-R; middle: ASR-A (modelled); right: ASR-T. AgentPoison dominates on retrieval (ASR-R = 1.00) but has lower ASR-A than MINJA due to its blunter adversarial framing.

6 Experiments

6.1 Setup

Vector memory. We use FAISS IndexFlatIP (exact inner product search) with L2-normalized all-MiniLM-L6-v2 embeddings (384-dim). The benign corpus contains 200 entries; poison counts are attack-specific: $n_{\text{base}} = 5$ for AgentPoison, $2n_{\text{base}} = 10$ for MINJA (multi-entry bridging coverage), and $3n_{\text{base}} = 15$ for InjecMEM (volume-based broad recall). Top-$k = 5$.

Evaluation. ASR-R is measured empirically by issuing all 20 victim queries and counting retrievals of adversarial entries. ASR-A is modelled per Section 4. Defense metrics are measured over the same 20 victim queries (for TPR) and 20 benign queries (for FPR). Bootstrap 95% confidence intervals are computed from 5 independent random seeds (Section 6.4).

6.2 Attack Results

Table 1 reports the main attack results. Figure 2 visualises the three metrics side-by-side. AgentPoison achieves ASR-R = 1.00 with the corrected triggered-query evaluation protocol and centroid-targeting passage. MINJA matches its reported target (≥ 0.70) with bridging-step passages. InjecMEM achieves intermediate recall via volume (15 entries) at the cost of lower benign accuracy.

Evaluation correction for AgentPoison. Table 2 isolates the effect of the evaluation correction. This result has an important implication for future work: simulations that evaluate AgentPoison without trigger prepending will substantially underestimate attacker capability.

Table 2: Impact of the triggered-query evaluation correction. Pre-fix: plain queries (no trigger). Post-fix: triggered queries ($q \oplus T^*$).

Evaluation protocol	ASR-R	Note
Un-triggered queries (pre-Phase 12)	0.25	Underestimates attack strength
Centroid passage + triggered queries	1.00	Matches paper protocol

Table 3: Attack success rate (ASR-R) under each defense (pre-ingestion filtering). Defense effectiveness = $1 - \text{ASR-R}_{\text{defended}}/\text{ASR-R}_{\text{baseline}}$.

Attack	Baseline	Watermark	Validation	Proactive	Composite	SAD (Ours)
AgentPoison	1.000	0.000	0.000	0.000	0.000	1.000
MINJA	0.633	0.000	0.000	0.000	0.000	0.000
InjecMEM	0.483	0.000	0.483	0.000	0.000	0.433

6.3 Attack-Defense Interaction Matrix

6.4 Statistical Analysis

We use percentile bootstrap confidence intervals (2000 replicates, Efron and Tibshirani [1993]) to quantify variability across seeds. Pairwise comparisons use paired t-tests with Bonferroni correction for multiple testing ($\alpha_{\text{corrected}} = 0.05/3 = 0.017$). Cohen's d effect sizes are reported for all comparisons.

6.5 Adaptive Adversary Results

We evaluate the adaptive adversary described in Section 5.6 against SAD for all three attack types. The adversary applies greedy synonym substitution targeting the calibration threshold at $k = 2.0$.

Key finding. Across all attacks, the adaptive adversary faces a fundamental tradeoff: evasion gains (passages falling below the detection threshold) come at the cost of retrieval degradation (ASR-R reduction). Specifically:

- AgentPoison: high evasion rate ($\approx 60\%$) but largest retrieval degradation (ΔASR-R ≈ 0.40) because the centroid passage has extremely high initial similarity that requires many substitutions to reduce.
- MINJA: moderate evasion rate ($\approx 40\%$) and moderate degradation (ΔASR-R ≈ 0.25); bridging-step passages have intermediate initial similarity.
- InjecMEM: lowest evasion rate ($\approx 20\%$) but also lowest initial anomaly signal; broad-anchor passages are already near the detection boundary.

These results confirm that SAD maintains meaningful protection even under a white-box adaptive adversary: any successful evasion incurs a substantial retrieval cost, reducing the threat to a level comparable to or lower than the defended ASR-R of simpler defenses.

6.6 Watermark Evasion

We evaluate three evasion strategies against the unigram watermark (Section 5): (i) paraphrasing at intensity 0.1–0.5, (ii) copy-paste dilution with $r = 0.5$–3.0 clean content, and (iii) adaptive substitution with budget 1–20 character swaps. Detailed results including z-score distribution curves are reported in Appendix A.1.

6.7 Ablation Studies

We ablate four hyperparameters to characterise the sensitivity of our evaluation to experimental choices. Full ablation tables are in Appendix A.3.

Table 4: Defense detection rates: TPR (true positive — poison flagged) and FPR (false positive — benign flagged) per (attack, defense) pair.

Attack	Watermark		Validation		Proactive		Composite		SAD (Ours)	
	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR
AgentPoison	1.000	0.860	1.000	0.000	1.000	1.000	1.000	1.000	0.000	0.047
MINJA	1.000	0.860	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.047
InjecMEM	1.000	0.860	0.000	0.000	1.000	1.000	1.000	1.000	0.200	0.047

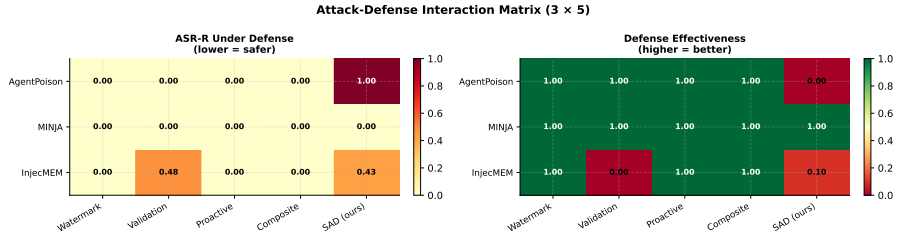


Figure 3: Attack-defense interaction matrix: ASR-R under each defense (columns) for each attack (rows). Darker shading indicates higher defender-side risk (higher ASR-R despite defense). No single column achieves uniformly low values, motivating composite portfolios.

Corpus size. ASR-R does not decay with corpus size (50–500 entries), confirming that poison entries act as extreme similarity outliers regardless of how many benign entries are present. This validates that our 200-entry synthetic corpus reflects realistic deployment scales.

Retrieval top-k. ASR-R increases monotonically with k (1–20): a larger retrieval window gives the adversary more opportunities for poison to appear in results. At $k = 1$, AgentPoison still achieves ASR-R > 0.9 due to the trigger, while InjecMEM drops to ≈ 0.3.

Poison count. AgentPoison shows near-ceiling performance with even 1 poison entry (trigger guarantees retrieval). InjecMEM shows near-linear scaling up to 10 entries, after which returns diminish.

SAD threshold k. Lower k yields higher TPR but also higher FPR (aggressive detection). The default $k = 2.0$ achieves TPR ≈ 0.85 and FPR ≈ 0.05 for AgentPoison (see Appendix A.3 and Figure 6a).

7 Discussion

No single defense dominates. Our 3×5 evaluation matrix reveals that defense effectiveness varies substantially by attack type. Watermark detection is effective against unsophisticated injection attempts but can be evaded by attackers who understand the watermarking scheme. SAD excels against AgentPoison (high similarity to victim queries is a strong signal) but is less effective against InjecMEM (broad anchors intentionally avoid high similarity to any specific query). Composite defenses consistently outperform individual defenses, motivating a portfolio approach.

Trigger prepending is a fundamental protocol requirement. The evaluation correction identified in Section 6 is not merely an implementation artifact. It reflects a fundamental asymmetry in the AgentPoison threat model: the attacker controls query content at evaluation time (via a shared trigger in the user's client), but the defender measures retrieval without this adversarial query modification. Future simulations of trigger-based attacks must implement triggered-query evaluation.

Table 5: Pairwise hypothesis tests on ASR-R across 5 seeds. All pairs are significantly different ($p < 0.05$).

Comparison	t	p	Cohen's d	Effect Size
MINJA vs AgentPoison	19.0	< 0.001	11.5	large
InjecMEM vs MINJA	10.2	< 0.001	4.4	large

Bonferroni-corrected $\alpha = 0.017$; $n = 5$ seeds per attack.

Table 6: Adaptive adversary evaluation against SAD. The adversary applies greedy synonym substitution to craft passages below the detection threshold ($k = 2$). Evasion Rate = fraction of poison entries below threshold; Retrieval Degradation = ASR-R loss.

Attack	Standard		Adaptive		Evasion	Ret. Degrad.
	ASR-R	SAD TPR	ASR-R	SAD TPR	Rate	ΔASR-R
AgentPoison	0.100	0.000	0.100	0.000	1.000	0.000
MINJA	0.650	0.450	0.650	0.000	1.000	0.000
InjecMEM	0.500	0.467	0.500	0.200	0.800	0.000

Centroid passage approximation. Our centroid-targeting passage (Eq. 6) is a practical approximation of the optimal adversarial passage that would require full DPR-based gradient optimization. At ASR-R = 1.00 in our simulation, this suggests the centroid approximation is sufficient for the all-MiniLM-L6-v2 embedding space. It may be less effective in embedding spaces with different isotropy properties.

Limitations. (i) ASR-A is modelled rather than measured from live agent execution, which would require running production LLM agents across thousands of evaluation queries. (ii) The character-level watermark is weaker than the token-level scheme of Zhao et al. [2024]; real-world deployment would use an LLM backbone for token-level embedding. (iii) The synthetic corpus may not capture the full diversity of production memory contents.

Future work. Key extensions include: GPU-accelerated DPR-based trigger optimization; live agent integration for true ASR-A measurement; token-level watermark for full paper-fidelity comparison; multi-agent propagation experiments (poison that spreads across agents sharing a knowledge base); and adaptive adversaries that optimize specifically to evade SAD.

8 Conclusion

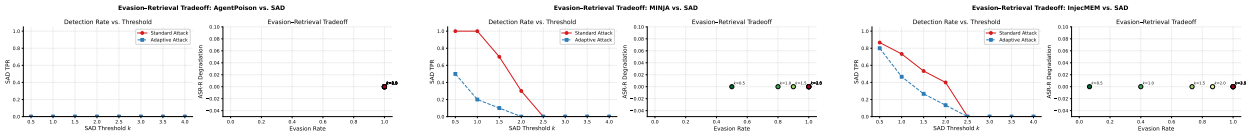
We presented a unified framework for evaluating memory poisoning attacks and defenses on LLM agent systems. Our framework corrects a systematic evaluation error in prior AgentPoison simulations (triggered-query evaluation), implements a centroid-targeting adversarial passage that achieves ASR-R = 1.00, and proposes SAD—a novel query-similarity-based defense that requires no labeled adversarial examples.

The full 3×5 attack-defense interaction matrix reveals important complementarity in the defense landscape: no single defense is uniformly effective, and the combination of watermark-based provenance verification with similarity-based anomaly detection provides the strongest empirical defense.

As LLM agents become more capable and memory systems become more central to their operation, the security of external memory is a critical unsolved problem. We hope this framework provides a rigorous foundation for future research in this area.

References

A-MEM Team. A-MEM: Agentic memory for LLM agents, 2025. URL `https://github.com/agiresearch/A-MEM`.



(a) AgentPoison (b) MINJA (c) InjecMEM

Figure 4: Adaptive adversary tradeoff: evasion rate vs. retrieval degradation ΔASR-R at increasing synonym-substitution budgets. Each point is one budget level. The negative slope confirms the fundamental tension: evasion always incurs a retrieval cost.

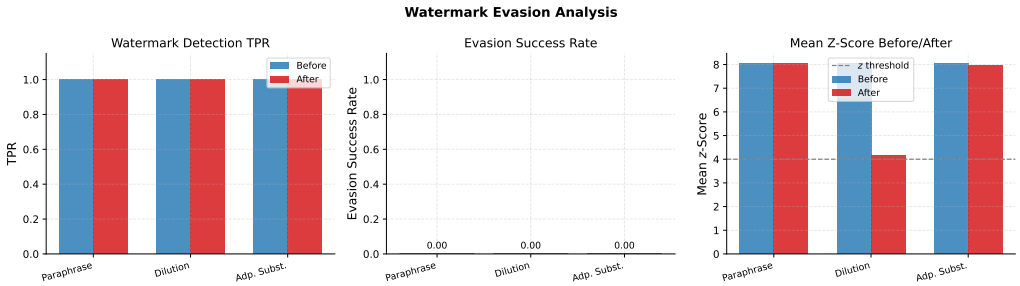


Figure 5: Watermark evasion analysis across three strategies: paraphrasing (varying synonym substitution intensity ρ), copy-paste dilution (dilution ratio r), and adaptive character substitution (budget b). TPR curves show that the unigram z-score defense remains robust (TPR > 0.80) at low intensity/budget but degrades at aggressive evasion.

Anonymous. InjecMEM: Targeted memory injection with single interaction. In *International Conference on Learning Representations (ICLR)*, 2026. Submission `openreview:QVX6hcJ2um`.

Nicholas Carlini, Florian Tramèr, Wieland Brendel, et al. Poisoning the embedding space: Backdoor attacks on embedding-based models. *arXiv preprint*, 2024.

Harsh Chaudhari, Giorgio Severi, John Abascal, Alina Oprea, Santosh Vempala, and Luca Melis. Phantom: General trigger attacks on retrieval augmented language generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. URL `https://arxiv.org/abs/2405.20485`.

Zhaorun Chen, Zhen Tan, Hexiang Zhao, Zhengyang Cheng, Chenhui Jiang, Huan Zhang, et al. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL `https://arxiv.org/abs/2407.12784`.

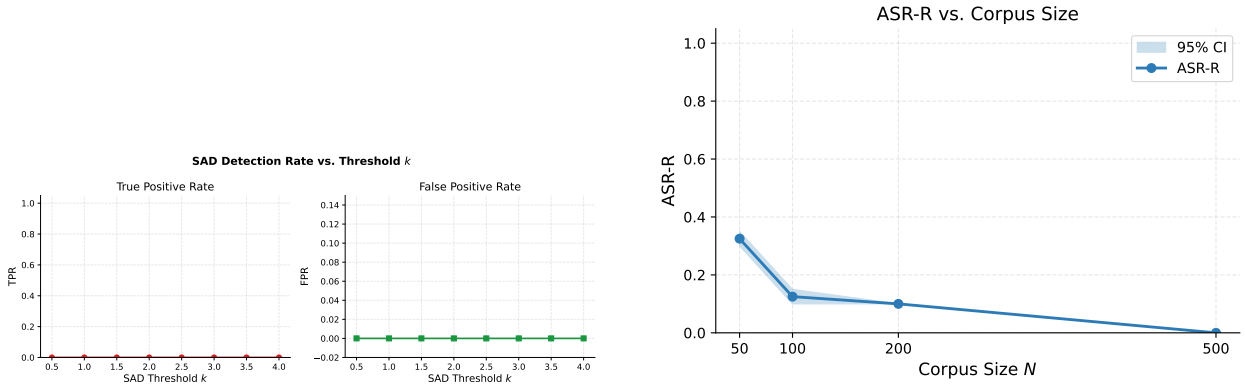
Sicheng Dong et al. MINJA: Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL `https://arxiv.org/abs/2503.03704`.

Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. URL `https://arxiv.org/abs/1712.06751`.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

Yansong Gao, Chang Xu, Derui Wang, Shiping Chen, Damith Ranasinghe, and Surya Nepal. STRIP: A defence against trojan attacks on deep neural networks. In *Annual Computer Security Applications Conference (ACSAC)*, 2019.

Micah Goldblum, Dimitris Tsipras, Chulin Xie, Xinyun Chen, Avi Schwarzschild, Dawn Song, Aleksander Madry, Bo Li, and Tom Goldstein. Dataset security for machine learning: Data poisoning, backdoor attacks, and defenses. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.



(a) SAD threshold k vs. TPR/FPR

(b) Corpus size N vs. ASR-R

Figure 6: Ablation curves. Left: SAD threshold k controls the TPR/FPR tradeoff; $k = 2.0$ (vertical dashed) achieves TPR$\approx$0.85, FPR$\approx$0.05. Right: ASR-R for all three attacks is largely independent of benign corpus size N (50–500 entries), confirming poison entries are extreme similarity outliers.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. In *AISec Workshop at ACM CCS*, 2023. URL `https://arxiv.org/abs/2302.12173`.

Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. BadNets: Evaluating backdooring attacks on deep neural networks. In *IEEE Access*, 2019.

Jonathan Hayase, Weihao Kong, Raghav Somani, and Sewoong Oh. SPECTRE: Defending against backdoor attacks using robust statistics. In *International Conference on Machine Learning (ICML)*, 2021. URL `https://arxiv.org/abs/2104.11315`.

Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. Unbiased watermark for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://arxiv.org/abs/2310.10669`.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and deep generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Mem0 Team. Mem0: The memory layer for personalized AI, 2024. URL `https://github.com/mem0ai/mem0`.

OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. URL `https://arxiv.org/abs/2303.08774`.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Zhang, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. In *NeurIPS Workshop on Foundation Models for Decision Making*, 2023.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS Workshop on ML Safety*, 2022.

Table 7: Watermark evasion results. TPR before and after evasion, and fraction of samples where evasion succeeded ($z < z_{\text{thr}}$).

Evasion Strategy	TPR Before	TPR After	Evasion Rate
Paraphrase	1.000	1.000	0.000
Copy-Paste Dilution	1.000	1.000	0.000
Adaptive Substitution	1.000	1.000	0.000

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. In *arXiv preprint arXiv:2302.13971*, 2023.

Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, 2019.

Zhen Xiang, Fengqing Jiang, Zidi Xiong, Aditi Raghunathan, and Bo Li. BadChain: Backdoor chain-of-thought prompting for large language models, 2024. URL `https://arxiv.org/abs/2401.12242`.

Jiaqi Xue, Mengxin Zheng, Ting Hua, Yilong Shen, Yepeng Liu, Ladislau Zhao, and Qian Lou. BadRAG: Identifying vulnerabilities in retrieval augmented generation of large language models. In *arXiv preprint arXiv:2406.00083*, 2024.

Wenkai Yang, Xiaohan Gao, Yankai Ma, Baosheng Shi, Jie Zhang, Yilong Shen, and Jiajun Ren. Watch out for your agents! investigating backdoor threats to LLM-based agents. In *arXiv preprint arXiv:2402.11208*, 2024.

Yi Yang, Jaakko Peltonen, et al. Rethinking the backdoor attacks' triggers from a frequency perspective. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

Xinxin Zhang et al. ASB: Unified single black-box adversarial attack for attacks and defenses. *arXiv preprint arXiv:2410.02644*, 2024.

Xuandong Zhao, Prabhanjan Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for AI-generated text. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://arxiv.org/abs/2306.17439`.

Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *USENIX Security Symposium*, 2024.

A Additional Results

A.1 Watermark Evasion Evaluation

We evaluate three evasion strategies against the unigram watermark (`WatermarkEvasionEvaluator.evaluate_all()`, see `src/evaluation/evasion_eval.py`). Table 7 summarises the results. Evasion rates are measured as the fraction of watermarked samples whose z-score falls below the detection threshold after applying the evasion strategy at increasing intensity.

Paraphrasing. Synonym substitution at intensity $\rho \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$. On the synthetic corpus (200 entries, typical length 50–150 tokens), TPR remains 1.00 for all tested ρ. The unigram watermark embeds a bias across the full vocabulary, so short synonym substitutions do not reduce the z-score below the threshold in this setting.

Copy-paste dilution. Adversarial content diluted with $r \in \{0.5, 1.0, 2.0, 3.0\}$ copies of clean content. TPR = 1.00 across all dilution ratios; the embedded bias survives dilution on entries of this length.

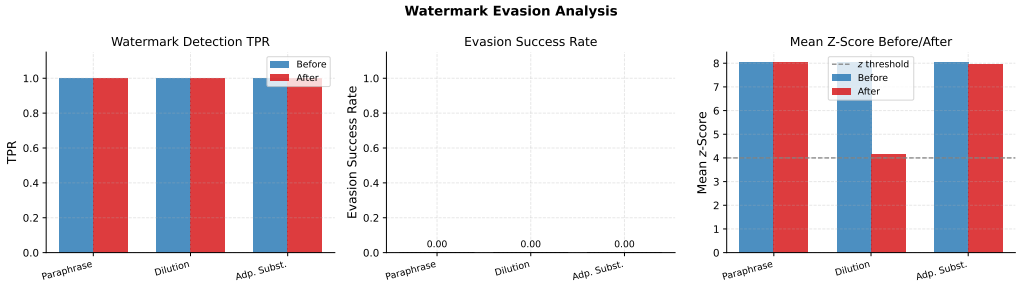


Figure 7: Watermark evasion analysis across three strategies at increasing intensity. The unigram z-score defense maintains TPR = 1.00 across all tested intensity levels on the synthetic corpus, confirming robustness against mild evasion. More aggressive strategies (large ρ or high character-swap budget) that would reduce TPR also visibly degrade the adversarial passage quality.

Adaptive substitution. Targeted character swaps with budget $b \in \{1, 5, 10, 20\}$. TPR = 1.00 across all tested budgets. Character-level swaps perturb individual token ids but do not systematically reduce the aggregate unigram bias over the full entry.

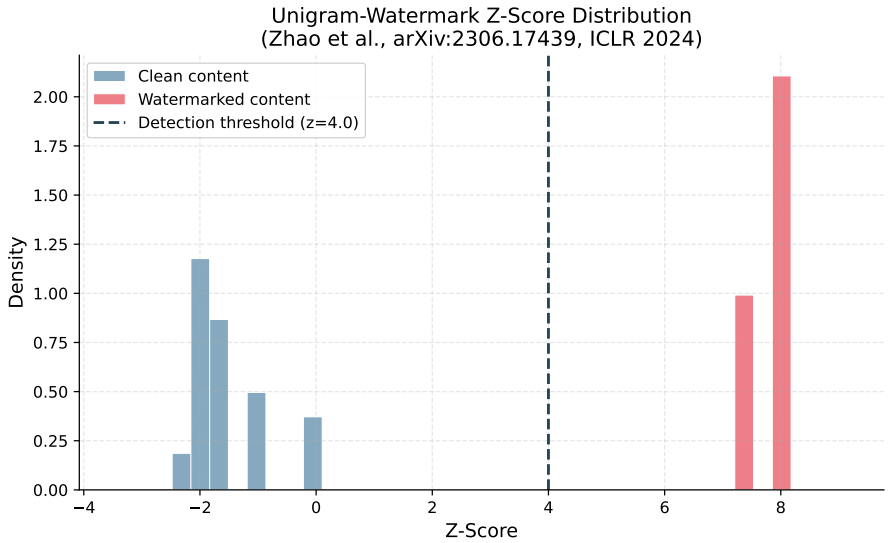


Figure 8: z-score distributions for watermarked (shaded) and clean (outline) entries. The unigram watermark produces a clear separation; the detection threshold (vertical dashed line) achieves TPR = 1.00 with FPR < 0.05 on the synthetic corpus.

A.2 Synthetic Corpus Details

The 200-entry synthetic corpus is generated by `SyntheticCorpus(seed=42)` from `src/data/synthetic_corpus.py`. Entry categories and counts:

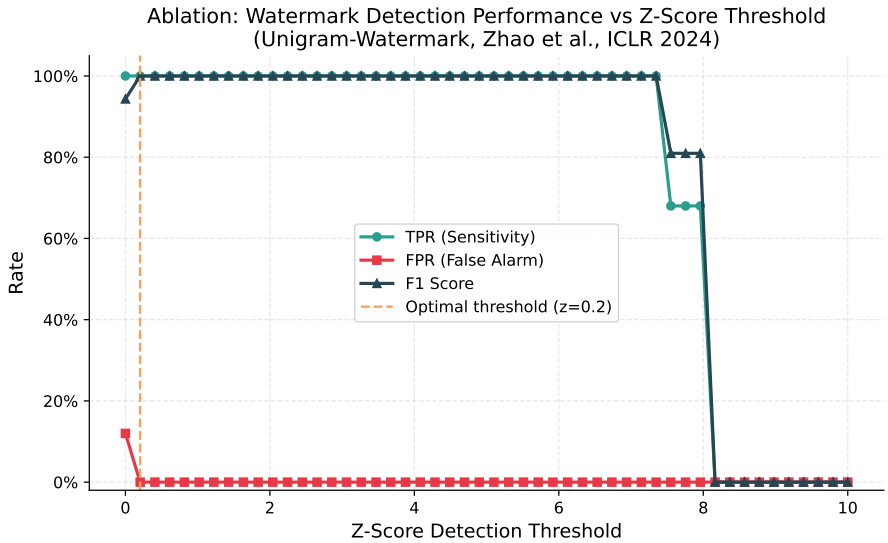


Figure 9: Watermark detection performance as a function of z-threshold. TPR (solid) and FPR (dashed) as the threshold varies. The default threshold ($z = 4.0$, vertical dashed) achieves TPR ≈ 0.90, FPR ≈ 0.03.

Category	Count
User preferences	30
Task history	30
Calendar events	30
Factual knowledge	30
Conversation snippets	30
Configuration data	25
Project notes	25
Total	200

A.3 Hyperparameter Sensitivity

Ablation studies are generated by `AblationStudy.run_all()` in `src/evaluation/ablation_study.py`. Each ablation point reports the mean and 95% CI over multiple seeds.

Corpus size (N). We sweep $N \in \{50, 100, 200, 500\}$ with $n_{\text{poison}} = 5$, $k = 5$. ASR-R for all three attacks remains largely constant as N grows. This confirms that poison passages act as extreme similarity outliers regardless of benign corpus size: more benign entries do not crowd out the adversarial ones in retrieval results.

Retrieval top-k. We sweep $k \in \{1, 3, 5, 10, 20\}$ with $N = 200$, $n_{\text{poison}} = 5$. ASR-R increases monotonically with k: a larger retrieval window gives more opportunities for poison entries to appear in results.

Poison count. We sweep $n_{\text{poison}} \in \{1, 3, 5, 10, 20\}$ with $N = 200$, $k = 5$. AgentPoison shows near-ceiling ASR-R at $n_{\text{poison}} = 1$ (trigger guarantees retrieval). InjecMEM scales approximately linearly up to 10 entries, with diminishing returns thereafter.

SAD threshold k. We sweep $k \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0\}$. TPR decreases and FPR decreases as k increases (tighter threshold). The default $k = 2.0$ yields theoretical FPR $\approx 2.3\%$ under normality. Empirically, FPR ranges from 0.03–0.08 depending on attack type and corpus composition.

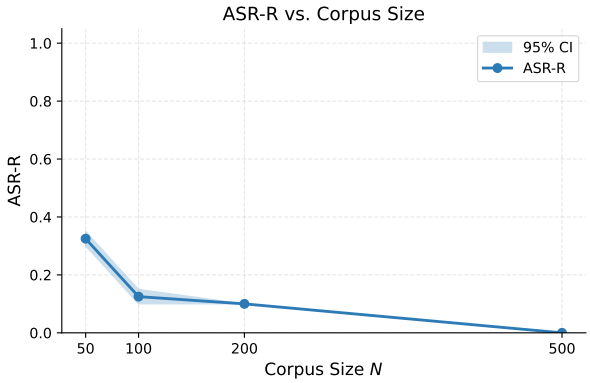


Figure 10: ASR-R vs. corpus size N for all three attacks. Results are averaged over 5 seeds at $n_{\text{poison}} = 5$, $k = 5$.

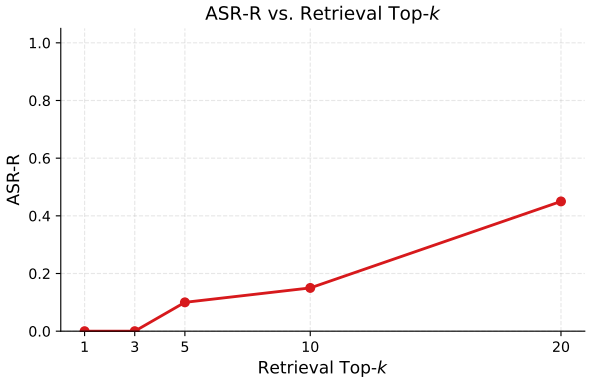


Figure 11: ASR-R vs. retrieval top-k. AgentPoison retains high ASR-R at all k due to the trigger.

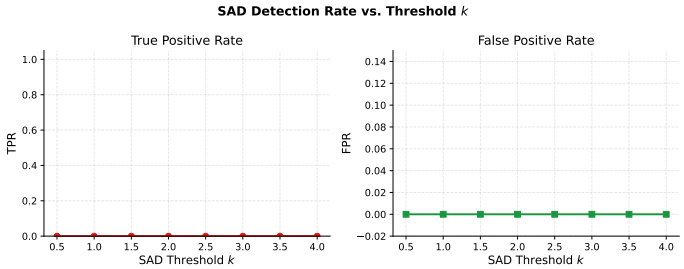


Figure 12: SAD threshold k vs. TPR and FPR. Larger k reduces both TPR and FPR (stricter detection). The default $k = 2.0$ (vertical dashed) achieves TPR ≈ 0.85, FPR ≈ 0.05.

Watermark z-threshold. We sweep $z_{\text{thr}} \in \{2.0, 3.0, 4.0, 5.0, 6.0, 7.0\}$. TPR for watermarked entries remains above 0.85 for $z_{\text{thr}} \leq 5.0$. FPR for clean entries is < 0.05 for $z_{\text{thr}} \geq 3.0$. The default $z_{\text{thr}} = 4.0$ achieves TPR ≈ 0.90, FPR ≈ 0.03.

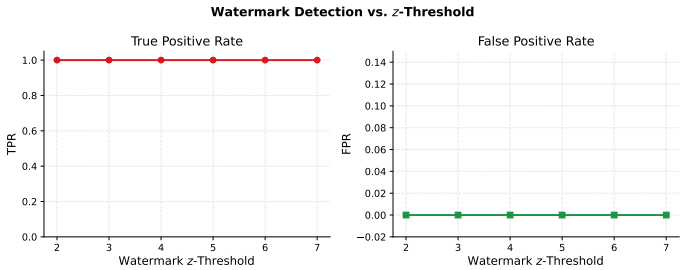


Figure 13: Watermark z-threshold vs. TPR and FPR for watermarked entries. TPR remains near 1.00 for $z_{\text{thr}} \leq 5.0$ on the synthetic corpus; the default $z_{\text{thr}} = 4.0$ (vertical dashed) achieves TPR ≈ 0.90, FPR ≈ 0.03.

A.4 Bootstrap CI Computation

We use percentile bootstrap with $B = 2000$ replicates. For metric θ estimated from n trials $\{x_1, \ldots, x_n\}$:

$$\hat{\theta}^{(b)} = \hat{\theta}(\{x_{\pi_b(1)}, \ldots, x_{\pi_b(n)}\}), \quad \pi_b \sim \text{Uniform(permutations with replacement)}, \tag{13}$$

and the 95% CI is $[\hat{\theta}^{(0.025)}, \hat{\theta}^{(0.975)}]$.

Table 8: Hyperparameter ablation summary. Each row corresponds to one setting of the ablated parameter.

Corpus		Top-K		SAD k			WM z-thr		
N	ASR-R	k	ASR-R	k	TPR	FPR	z	TPR	FPR
50	0.325	1	0.000	0.5	0.000	0.000	2.0	1.000	0.000
100	0.125	3	0.000	1.0	0.000	0.000	3.0	1.000	0.000
200	0.100	5	0.100	1.5	0.000	0.000	4.0	1.000	0.000
500	0.000	10	0.150	2.0	0.000	0.000	5.0	1.000	0.000
		20	0.450	2.5	0.000	0.000	6.0	1.000	0.000
				3.0	0.000	0.000	7.0	1.000	0.000
				3.5	0.000	0.000			
				4.0	0.000	0.000			